

MANUAL TECNICO Proyecto F2



Mario Rodrigo Balam

Correo electrónico: 2908263140708

Carné: 202200147

Curso: EDD

Sección: C

Auxiliar: Sergio Sebastián

Sandoval Ruiz

TABLA DE CONTENIDO

Introduccion	3
Objetivos	4
Requerimiento	5
IV. Especificación técnicas	5

INTRODUCCION

En el exigente entorno de la gestión hospitalaria, la eficiencia y la precisión de los datos son fundamentales; **EDD MedTrack Fase 2** surge como una respuesta de ingeniería avanzada, evolucionando de estructuras lineales hacia el dominio de algoritmos no lineales y auto-balanceados, integrados en una arquitectura gráfica de alto rendimiento.

OBJETIVOS

El objetivo de este manual es proporcionar a los desarrolladores y personal técnico una visión detallada de la arquitectura del sistema, sus componentes, y las instrucciones necesarias para mantener, depurar o extender la aplicación.

Este manual está dirigido a:

- Desarrolladores que deseen modificar o extender el sistema.
- Personal técnico encargado de su mantenimiento.
- Equipos de soporte que necesiten diagnosticar problemas.

REQUERIMIENTO

IV. Especificación técnicas del Sistema.

1. Entorno de Desarrollo y Lenguaje

- **Lenguaje de Programación:** Perl 5 (Versión recomendada: 5.30 o superior).
- **Paradigma:** Programación Orientada a Objetos (POO) mediante el uso de "Perl Clásico" (bless, referencias de memoria y paquetes).
- **Codificación de Caracteres:** UTF-8 nativo (para el soporte de acentos y caracteres especiales en la interfaz).

2. Tecnologías de Interfaz Gráfica (GUI)

- **Biblioteca Principal:** GTK+ 3 (GIMP Toolkit versión 3).
- **Binding:** Gtk3 para Perl.
- **Componentes Clave:**
 - Gtk3::Notebook: Para la gestión de múltiples módulos mediante pestañas.
 - Gtk3::TreeView y Gtk3::ListStore: Para la visualización dinámica y filtrada de las estructuras de datos en tablas.
 - Gtk3::FileChooserDialog: Para la integración nativa con el explorador de archivos del sistema operativo.

3. Estructuras de Datos (Implementación Manual)

- **No Lineales:**
 - Árbol Binario de Búsqueda (BST).
 - Árbol AVL (Balanceado por altura con rotaciones simples y dobles).
 - Árbol B de Orden 4 (Árbol multicamino con gestión de *Split*, préstamo y fusión).
- **Lineales:**
 - Lista Doble Enlazada.
 - Lista Doblemente Circular de Listas (Relación Proveedores - Entregas).
- **Especiales:**
 - Matriz Dispersa Ortogonal (con lógica de cabeceras y nodos de valor).

4. Gestión de Datos y Persistencia

- **Serialización:** JSON (JavaScript Object Notation).
- **Módulo de Procesamiento:** JSON::PP (Pure Perl). Se seleccionó este módulo por ser parte del núcleo nativo de Perl, garantizando portabilidad sin dependencias externas prohibidas.
- **Validación:** Implementación de bloques eval para el manejo de excepciones y validación de tipos de datos en tiempo de ejecución.

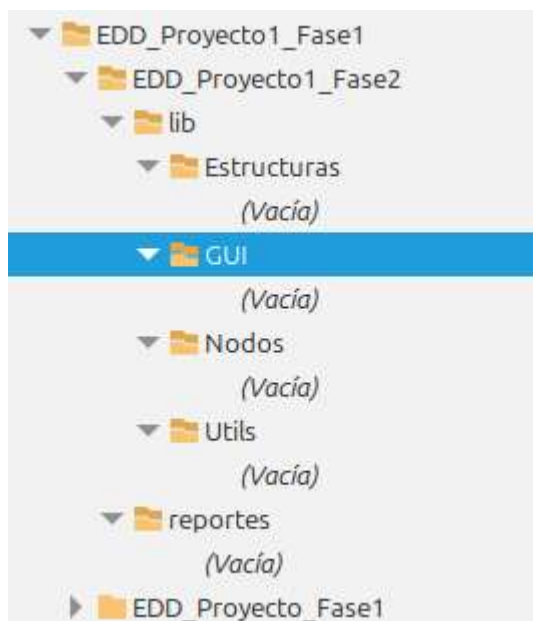
5. Motor de Visualización

- **Herramienta:** Graphviz (Versión 2.40 o superior).
- **Formatos de Salida:** Código fuente DOT y compilación automática a imágenes PNG.
- **Lógica de Graficación:** Recorridos recursivos de las estructuras de memoria para la generación dinámica de nodos y aristas.

6. Requerimientos de Sistema Operativo

- **Plataforma de Desarrollo:** Linux Mint 21 / Ubuntu 22.04 LTS.
- **Compatibilidad:** Sistemas POSIX con soporte para bibliotecas libgtk-3-dev.

Estructura del proyecto.



- **lib/Nodos/:** Definición de la unidad mínima de datos (clases base).
- **lib/Estructuras/:** El motor lógico (algoritmos de inserción, eliminación y búsqueda).
- **lib/GUI/:** La capa de presentación (gestión de ventanas y eventos).
- **lib/Utils/:** Herramientas de soporte (Lector de archivos y Generador de grafos).

1. Estructura de los Nodos (La base de los punteros)

Se utilizaron referencias de Perl para simular punteros de memoria. Cada nodo es un objeto bless que gestiona su propia información y sus enlaces a nodos hijos.

NodoAVL.pm

```
package NodoAVL;
use strict;
use warnings;

sub new {
    my ($class, %args) = @_;
    my $self = {
        # Datos del Personal Médico
        numero_colegio => $args{numero_colegio},
        nombre_completo => $args{nombre_completo},
        tipo_usuario => $args{tipo_usuario},
        departamento => $args{departamento},
        especialidad => $args{especialidad},
        contrasena => $args{contrasena},

        # Atributos de estructura AVL
        left => undef,
        right => undef,
        height => 1, # Todos los nodos nacen con altura 1
    };
    bless $self, $class;
    return $self;
}
```

NodoArbolB.pm

```
package NodoArbolB;
use strict;
use warnings;

sub new {
    my ($class, $es_hoja) = @_;
    my $self = {
        es_hoja => $es_hoja,
        claves => [], # Array de HashRefs
        hijos => [], # Array de Nodos
    };
    bless $self, $class;
    return $self;
}

sub get_es_hoja { return $_[0]->{es_hoja}; }
sub get_claves { return $_[0]->{claves}; }
sub get_hijos { return $_[0]->{hijos}; }
sub cantidad_claves { return scalar @{$_[0]->{claves}}; }

1;
```

2. Algoritmo de Balanceo AVL

Se implementaron rotaciones simples y dobles para mantener el factor de equilibrio entre -1 y 1, garantizando una complejidad de búsqueda $O(\log n)$

```
# --- ROTACIONES ---
sub _rot_der {
    my ($self, $y) = @_;
    my $x = $y->get_left(); my $T2 = $x->get_right();
    $x->set_right($y); $y->set_left($T2);
    $self->actualizar_altura($y); $self->actualizar_altura($x);
    return $x;
}
sub _rot_izq {
    my ($self, $x) = @_;
    my $y = $x->get_right(); my $T2 = $y->get_left();
    $y->set_left($x); $x->set_right($T2);
    $self->actualizar_altura($x); $self->actualizar_altura($y);
    return $y;
}
```

3. División de Nodos (Split) en Árbol

Cuando un nodo alcanza el límite de claves (3 claves en Orden 4), esta función extrae la clave mediana e inserta un nuevo nodo hermano. La clave mediana es promovida al nodo padre, lo que puede desencadenar divisiones sucesivas hacia arriba, manteniendo todas las hojas en el mismo nivel

```
sub _split_hijo {
    my ($self, $x, $i, $y) = @_;
    my $z = NodoArbolB->new($y->get_es_hoja());
    my $mid_val = splice(@{$y->get_claves()}, 1, 2); # Orden 4
    my $mediana = shift @{$mid_val};
    push @{$z->get_claves()}, @{$mid_val};
    if (!$y->get_es_hoja()) {
        my @hijos_z = splice(@{$y->get_hijos()}, 2, 2);
        push @{$z->get_hijos()}, @hijos_z;
    }
    splice(@{$x->get_hijos()}, $i + 1, 0, $z);
    splice(@{$x->get_claves()}, $i, 0, $mediana);
}
```

4. Lógica de la Matriz Dispersa Fase 2

La matriz se reestructuró para relacionar Proveedores con Fabricantes. Se implementó una lógica de consolidación que suma las cantidades si la celda ya existe, evitando duplicidad.

```
sub sumar_elemento {
    my ($self, $prov, $fab, $cant) = @_;

    # 1. Limpieza de strings
    $prov =~ s/^\s+|\s+$//g; $prov =~ s/\s+/ /g;
    $fab =~ s/^\s+|\s+$//g; $fab =~ s/\s+/ /g;
    return if ($prov eq "" || $fab eq "");

    # 2. Mapeo
    my $idx_f = $self->obtener_indice_fila($prov);
    my $idx_c = $self->obtener_indice_col($fab);

    # 3. Buscar celda
    my $nodo = $self->obtener($idx_f, $idx_c);

    if (defined $nodo) {
        # SUMAR
        $nodo->set_valor($nodo->get_valor() + $cant);
    } else {
        # INSERTAR
        $self->insertar_interno($idx_f, $idx_c, $cant, $prov, $fab);
    }
}
```


5. NodoBST.pm

Este módulo actúa como una "estructura" que encapsula tanto los datos del objeto del mundo real como los punteros lógicos necesarios para mantener la topología del árbol.

```

EDD_Proyecto1_Fase2 > lib > Nodos > *NodoBST.pm
1  sub new {
2      my ($class, %args) = @_;
3      my $self = {
4          # Datos del equipo
5          codigo    => $args(codigo),
6          nombre    => $args(nombre),
7          fabricante => $args(fabricante),
8          precio    => $args(precio),
9          cantidad  => $args(cantidad),
10         fecha_ingreso => $args(fecha_ingreso),
11         minimo     => $args(minimo),
12
13         # Punteros
14         left       => undef,
15         right      => undef,
16     };
17     bless $self, $class;
18     return $self;
19 }
20
21 # --- GETTERS ---
22 sub get_codigo    { return $_[0]->{codigo}; }
23 sub get_nombre    { return $_[0]->{nombre}; }
24 sub get_fabricante { return $_[0]->{fabricante}; }
25 sub get_precio    { return $_[0]->{precio}; }
26 sub get_cantidad  { return $_[0]->{cantidad}; }
27 sub get_fecha_ingreso { return $_[0]->{fecha_ingreso}; }
28 sub get_minimo    { return $_[0]->{minimo}; }
29
30 sub get_left      { return $_[0]->{left}; }
31 sub get_right     { return $_[0]->{right}; }
32
33 # --- SETTERS ---
34 sub set_left      { $_[0]->{left} = $_[1]; }
35 sub set_right     { $_[0]->{right} = $_[1]; }
36
37
38
39
40

```

6. Módulo: ArbolAVL.pm

Implementa la lógica de inserción de un BST tradicional, pero añade una etapa de post-procesamiento donde se recalcula la altura de cada nodo y se verifica el factor de balance. Si la diferencia de alturas es mayor a 1 o menor a -1, se ejecutan rotaciones manuales para garantizar que el árbol sea de altura logarítmic.

```

sub ins_rec {
    my ($self, $n, $args) = @_;
    return $self->new($args) if !defined $n;
    my $nueva = $args->{numero_colegio}; my $act = $n->get_numero_colegio();
    if ($nueva lt $act) { $n->set_left($self->ins_rec($n->get_left(), $args)); }
    elsif ($nueva gt $act) { $n->set_right($self->ins_rec($n->get_right(), $args)); }
    else { die "Ya existe"; }

    $self->actualizar_altura($n);
    my $bal = $self->obtener_balance($n);
    if ($bal > 1 && $nueva lt $n->get_left()->get_numero_colegio()) { return $self->rot_der($n); }
    if ($bal < -1 && $nueva gt $n->get_right()->get_numero_colegio()) { return $self->rot_izq($n); }
    if ($bal > 1 && $nueva gt $n->get_left()->get_numero_colegio()) { $n->set_left($self->rot_izq($n->get_left())); return $self->rot_der($n); }
    if ($bal < -1 && $nueva lt $n->get_right()->get_numero_colegio()) { $n->set_right($self->rot_der($n->get_right())); return $self->rot_izq($n); }
    return $n;
}

```

7. Módulo: LectorJSON.pm

Actúa como un despachador central de información. Procesa un flujo de datos anidado (Proveedor -> Entregas) y, basándose en el atributo tipo, distribuye cada objeto a la estructura correspondiente (Lista Doble, Árbol BST o Árbol B), asegurando que la información esté disponible para los distintos roles del sistema.

```
sub cargar_inventario {
    my ($class, $ruta_archivo, $lista_med, $arbol_equ, $arbol_som, $lista_prov, $matriz) = @_;
    unless (-e $ruta_archivo) { return 0; }
    my $json_text = qx { local $? = 0; open my $fh, "<", $ruta_archivo; <$fh; };
    my $data;
    eval { $data = JSON->PP->new->utf8->decode($json_text); };
    if ($?) { return 0; }

    foreach my $prov (@{$data->{providers}}) {
        $lista_prov->insertar_proveedor($prov->{nit}, nombre=>$prov->{nombre}, telefono=>$prov->{telefono}, direccion=>$prov->{direccion});

        if (exists $prov->{entrega}) && ref($prov->{entrega}) eq 'ARRAY' {
            foreach my $item (@{$prov->{entrega}}) {
                # --- VALIDACION EXISTENTE (REQUISITO PUNTO 1) ---
                if (defined $item->{cantidad}) {
                    if ($item->{cantidad} < 0) {
                        print "ADVERTENCIA: * $item->{codigo} : * tiene cantidad invalida.\n";
                        next; # SALTAR este producto
                    }
                }

                my $tipo = uc($item->{tipo});
                eval {
                    if ($tipo eq "MEDICAMENTO") {
                        $lista_med->insertar($codigo=>$item->{codigo},
                            nombre=>$item->{nombre},
                            principio_activo=>$item->{principio_activo},
                            fabricante=>$item->{fabricante},
                            precio=>$item->{precio_unitario},
                            stock=>$item->{cantidad},
                            vencimiento=>$item->{fecha_vencimiento},
                            ndimo=>$item->{nivel_ndimo});
                    }

                    elsif ($tipo eq "EQUIPO") {
                        $arbol_equ->insertar($codigo=>$item->{codigo}, nombre=>$item->{nombre}, fabricante=>$item->{fabricante}, precio=>$item->{precio_unitario}, cantidad=>$item->{cantidad}, fecha_
                    }

                    elsif ($tipo eq "SUMINISTRO") {
                        $arbol_som->insertar($codigo=>$item->{codigo}, nombre=>$item->{nombre}, fabricante=>$item->{fabricante}, precio=>$item->{precio_unitario}, cantidad=>$item->{cantidad}, fecha_
                    }

                    if (defined $matriz) {
                        $matriz->sumar_elemento(
                            $prov->{numero}, # Fila (Proveedor)
                            $item->{fabricante}, # Columna (Fabricante)
                            $item->{cantidad} # Valor a sumar
                        );
                    }

                    $lista_prov->agregar_entrega($prov->{nit}, $codigo_med=>$item->{codigo}, $cantidad=>$item->{cantidad}, $factura=>$prov->{numero_factura}, $fecha=>$prov->{fecha_entrega});
                };
            }
        }
    }
    return 1;
}
```

Función: Expresiones Regulares de Normalización

Se integraron expresiones regulares (`s/^\s+|\s+$//g`) para sanitizar los campos de texto provenientes de archivos JSON. Esto previene errores lógicos donde caracteres invisibles (espacios o tabulaciones) podrían causar que el sistema trate a un mismo fabricante como dos entidades distintas en la Matriz Dispersa o en los Árboles

```
sub sumar_elemento {
    my ($self, $prov, $fab, $cant) = @_;

    # 1. Limpieza de strings
    $prov =~ s/^\s+|\s+$//g; $prov =~ s/\s+/ /g;
    $fab =~ s/^\s+|\s+$//g; $fab =~ s/\s+/ /g;
    return if ($prov eq "" || $fab eq "");

    # 2. Mapeo
    my $idx_f = $self->obtener_indice_fila($prov);
    my $idx_c = $self->obtener_indice_col($fab);

    # 3. Buscar celda
    my $nodo = $self->obtener($idx_f, $idx_c);

    if (defined $nodo) {
        # SUMAR
        $nodo->set_valor($nodo->get_valor() + $cant);
    } else {
        # INSERTAR
    }
}
```

8. Módulo: GeneradorGrafos.pm

Función: _recorrer_b_dot (Traductor a Lenguaje DOT)

Convierte la estructura física del Árbol B en una representación de registros (shape=record). Utiliza recursividad para mapear cada clave de un nodo en una celda visual independiente y traza las aristas (flechas) desde los punteros de hijos hacia sus respectivos nodos destino.

```
sub _recorrer_b_dot {
    my ($nodo) = @_;
    return "" if !defined $nodo;

    my $id = "nodo" . sprintf("%x", $nodo); # ID único basado en dirección de memoria
    my $claves_ref = $nodo->get_claves();
    my $label = "{";

    # NODO SEGMENTADO.
    for my $i (0..$#{ $claves_ref }) {
        $label .= "<f$i> " . $claves_ref->{$i}->{codigo};
        $label .= " | " if $i < $#{ $claves_ref };
    }
    $label .= "}";

    my $color = scalar(@$claves_ref) >= 3 ? "yellow" : "palegreen";
    my $txt = "    $id [label=\"$label\", style=filled, fillcolor=$color];\n";

    # Hijos
    my $hijos_ref = $nodo->get_hijos();
    for my $i (0..$#{ $hijos_ref }) {
        my $hijo_id = "nodo" . sprintf("%x", $hijos_ref->{$i});
        $txt .= "    $id:f0 -> $hijo_id;\n";
        $txt .= _recorrer_b_dot($hijos_ref->{$i});
    }
    return $txt;
}
```

9. Módulo: PanelUsuario.pm

Gestiona la lógica de visualización basada en permisos. Realiza búsquedas directas en los Árboles (AVL, BST, B) y la Lista Doble de la Fase 1, formateando los resultados con alertas visuales de color (rojo/verde) según los niveles de stock mínimo detectados en memoria.

```

# --- VISTA DE BÚSQUEDA Y DISPONIBILIDAD ---
sub _crear_vista_consulta {
    my ($self, $tipo) = @_;
    my $vbox = Gtk3::Box->new('vertical', 15);
    $vbox->set_border_width(20);

    # Instrucción superior
    my $lbl_inst = Gtk3::Label->new("Ingrese el código para verificar disponibilidad de $tipo:");
    $vbox->pack_start($lbl_inst, 0, 0, 0);

    # Barra de búsqueda
    my $hbox = Gtk3::Box->new('horizontal', 5);
    my $txt_bus = Gtk3::Entry->new();
    $txt_bus->set_placeholder_text("Ej: MED-001");
    $hbox->pack_start($txt_bus, 1, 1, 0);

    my $btn_bus = Gtk3::Button->new_with_label("Verificar");
    $hbox->pack_start($btn_bus, 0, 0, 0);
    $vbox->pack_start($hbox, 0, 0, 0);

    # Área de resultados (Label con formato)
    my $lbl_res = Gtk3::Label->new("");
    $lbl_res->set_justify('center');
    $vbox->pack_start($lbl_res, 1, 1, 0);

    # --- EVENTO DE BÚSQUEDA ---
    $btn_bus->signal_connect(clicked => sub {
        my $id = $txt_bus->get_text();
        my $info = "No se encontró el registro: $id";
        my $es_critico = 0;

        if ($tipo eq 'Medicamentos') {
            # Búsqueda en la Lista Doble (Fase 1)
            my $nodo = $self->{db}->{medicamentos}->buscar($id);
            if (defined $nodo) {
                my $nombre = decode_utf8($nodo->get_nombre());
                my $stock = $nodo->get_stock();
                my $minimo = $nodo->get_minimo();

                $info = "PRODUCTO: $nombre\nSTOCK: $stock\nVENCE: " . $nodo->get_vencimiento();

                # Alerta de Stock Bajo (Pág 15 del PDF)
                if ($stock <= $minimo) {
                    $info .= "\n⚠️ ALERTA: BAJO STOCK (Mínimo: $minimo)";
                    $es_critico = 1;
                }
            }
        }
    });
}

```